

Formula-Based Time Complexity Optimization

1 Mathematical Formulas

The following mathematical formulas can be used to replace loop-based computations and reduce the time complexity.

Sum	Formula
Sum of first n natural numbers	$\sum_{i=1}^n i = \frac{n(n+1)}{2}$
Sum of squares of first n natural numbers	$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$
Sum of cubes of first n natural numbers	$\sum_{i=1}^n i^3 = \left[\frac{n(n+1)}{2} \right]^2$
Sum of first n even natural numbers	$2 + 4 + 6 + \dots + 2n = n(n+1)$
Sum of first n odd natural numbers	$1 + 3 + 5 + \dots + (2n-1) = n^2$

2 Sum of First n Natural Numbers

We need to calculate:

$$1 + 2 + 3 + \dots + n$$

2.1 For-Loop Approach

```
int sum = 0;

for(int i = 1; i <= n; i++)
{
    sum = sum + i;
}
```

2.2 Step 1: Input Parameter

The input parameter is:

$$n = \text{number of natural numbers}$$

2.3 Step 2: Number of Loop Executions

The loop executes for:

$$i = 1, 2, 3, \dots, n$$

Lower limit:

$$l = 1$$

Upper limit:

$$u = n$$

Using:

$$\text{Number of iterations} = u - l + 1$$

we get:

$$n - 1 + 1 = n$$

Therefore,

$$\text{Number of iterations} = n$$

2.4 Step 3: Basic Operation

The basic operation is:

```
sum = sum + i;
```

This operation is performed once during every iteration.

Therefore,

$$C(n) = \sum_{i=1}^n n - 1 + 1$$

$$C(n) = n$$

Hence,

$$T(n) = O(n)$$

2.5 Step 4: Optimization Using Formula

We know:

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

Therefore, the optimized code is:

```
int sum = n * (n + 1) / 2;
```

There is no loop.
Therefore,

$$T(n) = O(1)$$

2.6 Comparison

Method	Time Complexity
For-loop	$O(n)$
Formula	$O(1)$

3 Sum of Squares of First n Natural Numbers

We need to calculate:

$$1^2 + 2^2 + 3^2 + \dots + n^2$$

3.1 For-Loop Approach

```
int sum = 0;

for(int i = 1; i <= n; i++)
{
    sum = sum + i * i;
}
```

3.2 Step 1: Input Parameter

The input parameter is:

$$n$$

3.3 Step 2: Number of Loop Executions

Lower limit:

$$l = 1$$

Upper limit:

$$u = n$$

Using:

$$u - l + 1$$

we get:

$$n - 1 + 1 = n$$

Therefore,

$$\boxed{\text{Number of iterations} = n}$$

3.4 Step 3: Basic Operation

The basic operation is:

```
sum = sum + i * i;
```

Therefore,

$$C(n) = \sum_{i=1}^n n - 1 + 1$$

$$C(n) = n$$

Hence,

$$\boxed{T(n) = O(n)}$$

3.5 Step 4: Optimization Using Formula

We know:

$$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

Therefore, the optimized code is:

```
int sum = n * (n + 1) * (2 * n + 1) / 6;
```

There is no loop.

Hence,

$$\boxed{T(n) = O(1)}$$

4 Sum of Cubes of First n Natural Numbers

We need to calculate:

$$1^3 + 2^3 + 3^3 + \dots + n^3$$

4.1 For-Loop Approach

```
int sum = 0;

for(int i = 1; i <= n; i++)
{
    sum = sum + i * i * i;
}
```

4.2 Step 1: Input Parameter

The input parameter is:

$$n$$

4.3 Step 2: Number of Loop Executions

Lower limit:

$$l = 1$$

Upper limit:

$$u = n$$

Using:

$$u - l + 1$$

we get:

$$n - 1 + 1 = n$$

Therefore,

$$\boxed{\text{Number of iterations} = n}$$

4.4 Step 3: Basic Operation

The basic operation is:

```
sum = sum + i * i * i;
```

Therefore,

$$C(n) = \sum_{i=1}^n n - 1 + 1$$

$$C(n) = n$$

Hence,

$$\boxed{T(n) = O(n)}$$

4.5 Step 4: Optimization Using Formula

We know:

$$\sum_{i=1}^n i^3 = \left(\frac{n(n+1)}{2} \right)^2$$

Therefore, the optimized code is:

```
int sum = (n * (n + 1) / 2) *
          (n * (n + 1) / 2);
```

There is no loop.

Hence,

$$T(n) = O(1)$$

5 Sum of First n Even Natural Numbers

The first n even natural numbers are:

$$2, 4, 6, 8, \dots, 2n$$

We need to calculate:

$$2 + 4 + 6 + \dots + 2n$$

5.1 For-Loop Approach

```
int sum = 0;

for(int i = 1; i <= n; i = i + 1)
{
    sum = sum + 2*i;
}
```

5.2 Step 1: Input Parameter

The input parameter is:

$$n$$

Hence,

$$\text{Number of iterations} = n$$

5.3 Step 3: Basic Operation

The basic operation is:

```
addition and multiplication.
```

Therefore,

$$C(n) = \sum_{i=1}^n n - 1 + 1$$

$$C(n) = n$$

Hence,

$$T(n) = O(n)$$

5.4 Step 4: Optimization Using Formula

We know:

$$2 + 4 + 6 + \dots + 2n = n(n + 1)$$

Therefore, the optimized code is:

```
int sum = n * (n + 1);
```

There is no loop.

Hence,

$$T(n) = O(1)$$

6 Sum of First n Odd Natural Numbers

The first n odd natural numbers are:

$$1, 3, 5, 7, \dots, (2n - 1)$$

We need to calculate:

$$1 + 3 + 5 + \dots + (2n - 1)$$

6.1 For-Loop Approach

```
int sum = 0;

for(int i = 1; i <= n; i = i + 1)
{
    sum = sum + 2*i-1;
}
```

6.2 Step 1: Input Parameter

The input parameter is:

$$n$$

6.3 Step 2: Number of Loop Executions

$$= n$$

Therefore,

$$\text{Number of iterations} = n$$

6.4 Step 3: Basic Operation

The basic operation is:

addition and multiplication

Therefore,

$$C(n) = \sum_{i=1}^n n - 1 + 1$$

$$C(n) = n$$

Hence,

$$T(n) = O(n)$$

6.5 Step 4: Optimization Using Formula

We know:

$$1 + 3 + 5 + \cdots + (2n - 1) = n^2$$

Therefore, the optimized code is:

```
int sum = n * n;
```

There is no loop.

Hence,

$$T(n) = O(1)$$

7 Overall Comparison

Problem	Loop Iterations	Loop Complexity	Optimized Complexity
Natural numbers	n	$O(n)$	$O(1)$
Squares	n	$O(n)$	$O(1)$
Cubes	n	$O(n)$	$O(1)$
Even numbers	n	$O(n)$	$O(1)$
Odd numbers	n	$O(n)$	$O(1)$

8 Key Observation

A loop-based solution repeatedly performs the same basic operation.

For example:

$$C(n) = \sum_{i=1}^n n - 1 + 1 = n$$

Therefore:

$$T(n) = O(n)$$

However, if a mathematical formula is available, the entire loop can be replaced by a constant number of arithmetic operations.

Thus:

Loop-based solution	$\longrightarrow$	$O(n)$
---------------------	-------------------	--------

whereas

Formula-based solution	$\longrightarrow$	$O(1)$
------------------------	-------------------	--------

The key optimization principle is:

Recognize the mathematical pattern and eliminate the loop.
--